# Lab 2: Building Desktop Applications with Multiple Forms in PyQt6

CS355/CE373 Database Systems
Fall 2026


**Dhanani School of Science and Engineering**
Habib University

# Contents

# 1 Instructions

- This lab will contribute ≈1% towards the final grade.
- The deadline for this lab is one week after your lab ends. **However, you must also submit all the work completed in the lab at the end of the lab.**
- The lab must be submitted online via CANVAS. You are required to submit a zip file that contains both *.py* and *.ui* files.
- The zip file should be named as *Lab_02_aa1234.zip* where *aa1234* will be replaced with your student id.
- **Files that don't follow the appropriate naming convention will not be graded.**

## 1.1 Marking scheme

This lab will be marked out of 100.

- 50 marks: Lab completion.
- 50 marks: Progress and attendance (validated via viva).

## 1.2 Late submission policy

No late submissions are allowed for this lab.

# 2 Objective

The objective of this lab is to enable students to work with multiple GUI forms in their application and exchange data across them. This lab will cover an example that will demonstrate how to transfer data between two forms. The main task of this lab is to build a library management system.

## 3 Example: Transferring the Data Between Multiple Forms

In this example, we will consider an application that consists of two forms: MainForm and ViewForm. Once the application is started, the MainForm will load. In the MainForm, the student will enter the Name and ID, and then click on submit. Once the submit button is clicked, the GPA and Major of the student along with Name and ID, will be displayed in the ViewForm which is a read-only form.

In this application, the data will be fetched from a hardcoded Python List containing 3 records which can be seen in Table 1

| Name | ID | Major | GPA |
|---|---|---|---|
| Ahmed | 4289 | CS | 3.85 |
| Hammad | 4305 | CS | 3.53 |
| Mohsin | 4333 | CS | 3.92 |

Table 1: Student Entries

In the MainForm, the ID will be a combo box field. Once the ID is selected, the name text box will be populated. Furthermore, once the submit button is clicked all the corresponding details will be transferred to the ViewForm, which will then display them in a view-only form. The MainForm and ViewForm can be viewed in Figure 1

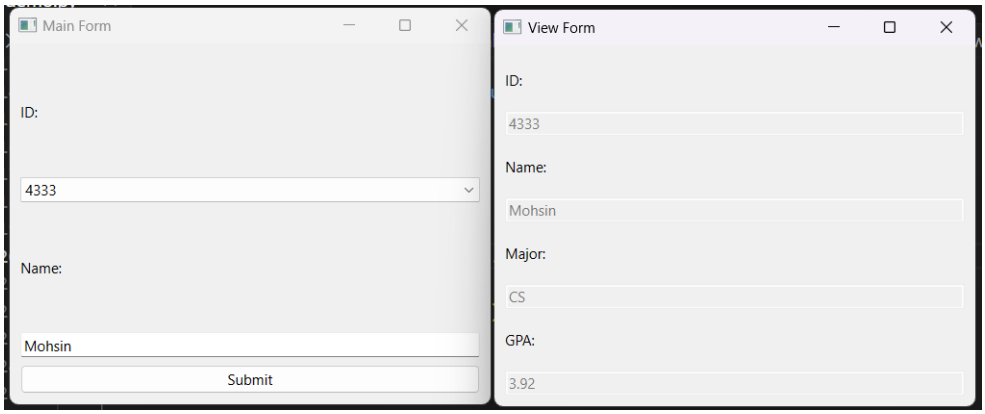


Figure 1: Main Form & View Form in PyQt6

The code and ui for this sample application can be found in the **Lab 2 - Building Desktop applications with multiple forms in PyQt6** module of your lab section on LMS under the **Example** header.

# 4 Exercise

For the following exercise, you are provided 3 files that can be accessed from **Lab 2 - Building Desktop applications with multiple forms in PyQt6** module of your lab section on LMS under the **Exercise** header. You are required to work with them only.

- `Lab02.ui` (no need to edit, this is the main form)
- `view.ui` (no need to edit, this is the view form)
- `app.py` (This is the skeleton file which you will edit. Only make changes where it is indicated by `...`)

In this exercise, you have to develop a Library Management System that will allow the users to search for books by providing criteria, view the details of a particular book, and delete the book from the system.

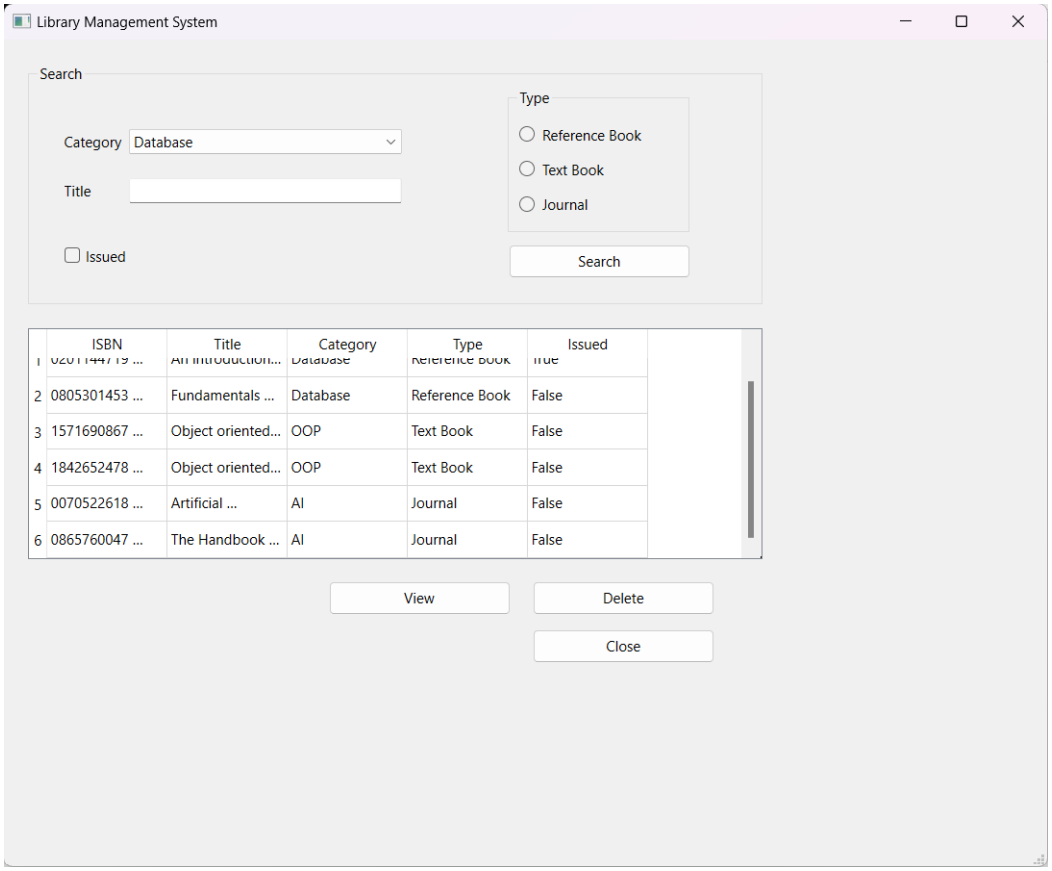


Figure 2: Library Management System using PyQt6

The Book Search Form will have the following functionality.

1. When the form is loaded, all the books which are stored in the **Python List** will be displayed in the table widget. The table widget has the following columns:
   - ISBN
   - Title
   - Category

- Type
- Issued

2. The Category combo box will show the following options:
   - Database
   - OOP
   - AI
3. Users can enter any search criteria, such as Category, Type, Title, and Issued, and then click 'Search' to find books that match those criteria. Only the books that match the search criteria will be loaded in the **table** widget. For example, if the Category is **Database**, the Title field is empty, the Type is Reference Book and the Issued Checkbox is Checked, the following will be the search result.

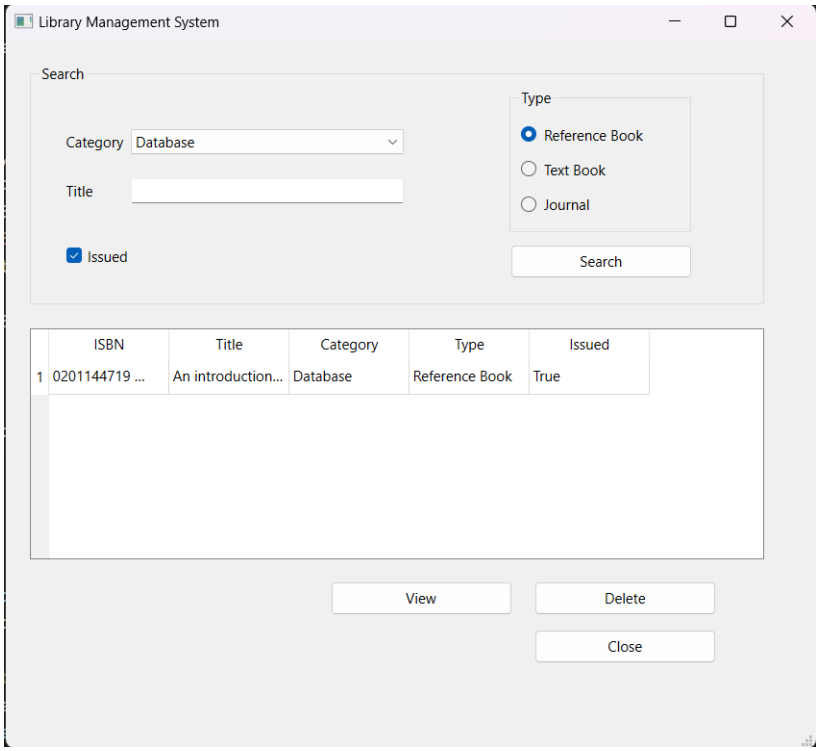


Figure 3: Search Output

4. The ‘Delete’ button will request the user for confirmation of deletion as follows:

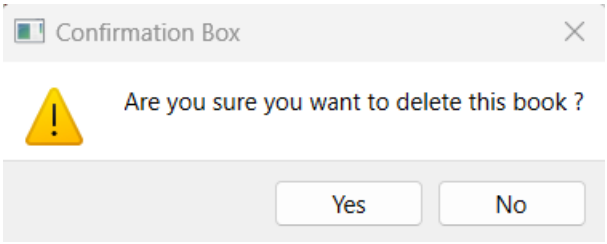


Figure 4: Confirmation Window for Deleting a Book

The book will be deleted if you choose ‘Yes’, but choosing ‘No’ will keep it in the system.

5. The view button will redirect the user to the View Book form, where the book’s details will be displayed. The View button will open the View Book form on top of the Search Book form. In addition, the book details will be displayed in view-only mode and the user cannot edit them.

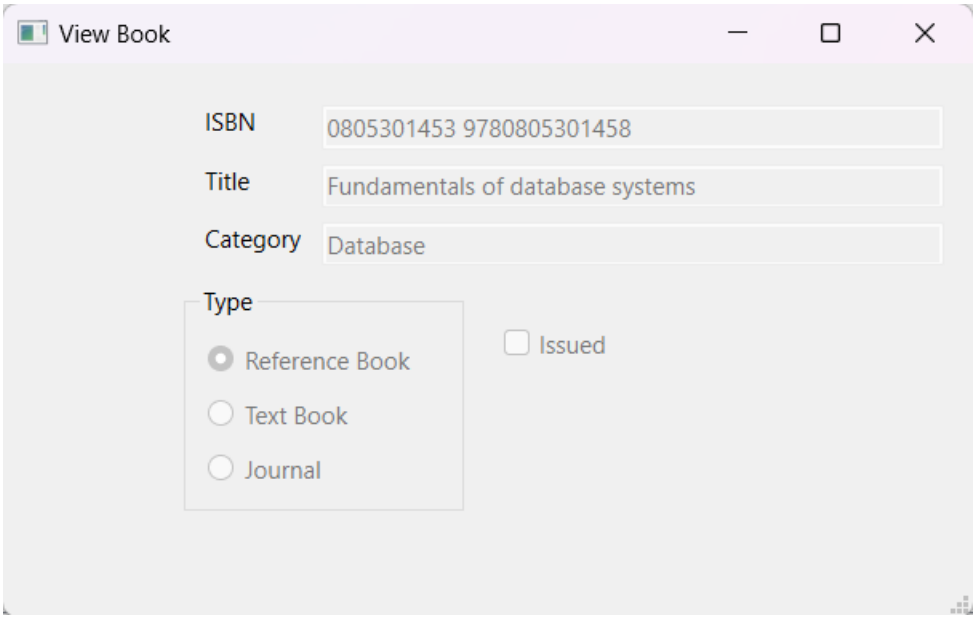


Figure 5: View Book Form in PyQt6

6. The application will close upon clicking the close button.

Since you have not yet studied database connectivity, the details of all the books are hardcoded in a Python List **books**. You are required to work on the same file (`app.py`) in this lab and implement the following functions:

1. The **search** function implements the search functionality. In this function, the user selection is gathered from the Title, Category, Type, and Issued fields. Afterward, the data is filtered on the basis of the user selection. Finally, the filtered data is then displayed on the table widget.
2. The **view** function implements the view book functionality. It first checks if a row in the table widget is selected, else it does not do anything. If a row is selected, the

book details are extracted from the row and then passed on to the ViewBook Form and then displayed.

3. The **delete** function implements the delete book functionality. It first checks if a row in the table widget is selected, else it does not do anything. If a row is selected, then it asks the user for confirmation. If the user says yes, then the book is deleted from the **books** Python List, and all the book details are re-displayed on the table widget.
4. The **close** function implements the close form functionality. Once the close button is clicked, the entire application is closed.

**Note: In order for your skeleton file `app.py` to execute successfully, you need to ensure that the UI files are named exactly ("Lab02.ui") and ("view.ui") and the objectName of the table widget is (booksTableWidget)**